

ANN and MID Practical Codes

ANN Practical 1 - Activation Functions

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)

sigmoid = 1 / (1 + np.exp(-x))
relu = np.maximum(0, x)
tanh = np.tanh(x)
leaky_relu = np.where(x > 0, x, 0.01 * x)

exp_x = np.exp(x)
softmax = exp_x / np.sum(exp_x)

plt.plot(x, sigmoid, label='Sigmoid')
plt.plot(x, relu, label='ReLU')
plt.plot(x, tanh, label='Tanh')
plt.plot(x, leaky_relu, label='Leaky ReLU')
plt.plot(x, softmax, label='Softmax')

plt.legend()
plt.grid(True)
plt.show()
```

ANN Practical 2 - XOR using Backpropagation

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[0]])

model = Sequential()
model.add(Dense(4, input_dim=2, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(loss='binary_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])

model.fit(X, Y, epochs=1000, verbose=0)

print(model.predict(X))
```

ANN Practical 3 - Feed Forward Neural Network

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([[0],[1],[1],[1]])

model = Sequential()
model.add(Dense(3, input_dim=2, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

model.fit(X, Y, epochs=500, verbose=0)
```

```
print(model.predict(X))
```

ANN Practical 4 - Perceptron Learning Algorithm

```
import numpy as np

X = np.array([[0,0],[0,1],[1,0],[1,1]])
Y = np.array([0,0,0,1])

weights = np.zeros(2)
bias = 0

learning_rate = 0.1
epochs = 10

for epoch in range(epochs):
    for i in range(len(X)):
        summation = np.dot(X[i], weights) + bias
        output = 1 if summation >= 0 else 0

        error = Y[i] - output

        weights = weights + learning_rate * error * X[i]
        bias = bias + learning_rate * error

print(weights)
print(bias)
```

MID Practical 1 - Titanic Dataset Preprocessing

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('titanic.csv')

print(df.head())

print(df.isnull().sum())

df['Age'].fillna(df['Age'].mean(), inplace=True)

scaler = StandardScaler()

df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

print(df.head())
```

MID Practical 2 - Sentiment Analysis using KNN

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

texts = [
    'I love this movie',
    'This is amazing',
    'Very bad experience',
    'I hate this',
    'Excellent work',
    'Worst product'
]

labels = [1,1,0,0,1,0]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, labels, test_size=0.3
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print(accuracy_score(y_test, pred))
```

MID Practical 3 - Positive/Negative Sentiment Classification

```
positive_words = ['good', 'excellent', 'amazing', 'happy', 'love']
negative_words = ['bad', 'worst', 'hate', 'poor', 'sad']

text = input('Enter sentence: ').lower()

positive_count = 0
negative_count = 0

words = text.split()

for word in words:
    if word in positive_words:
        positive_count += 1
    elif word in negative_words:
        negative_count += 1

if positive_count > negative_count:
    print('Positive Sentiment')
elif negative_count > positive_count:
    print('Negative Sentiment')
else:
    print('Neutral Sentiment')
```

MID Practical 4 - Abalone Dataset using Linear Regression

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data'

columns = ['Sex', 'Length', 'Diameter', 'Height',
           'Whole_weight', 'Shucked_weight',
           'Viscera_weight', 'Shell_weight', 'Rings']

df = pd.read_csv(url, names=columns)

df['Sex'] = df['Sex'].map({'M':0, 'F':1, 'I':2})

X = df.drop('Rings', axis=1)
y = df['Rings']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)

model = LinearRegression()
model.fit(X_train, y_train)

pred = model.predict(X_test)

print(mean_squared_error(y_test, pred))
```
